

LLMOrchestrator

LLMOrchestrator

Her bağısız CLI kodlama aracı için tek bir kontrol düzlemi.

Özet

LLMOrchestrator, bağısız CLI ajanlarını (OpenCode, Claude Code, Gemini, Junie, Qwen Code) oluşturmak, yönetmek ve hibrit boru+dosya protokolü üzerinden iletişim kurmak için bağısız, yeniden kullanılabilir bir Go modülüdür. Ajan başına devre kesiciler, takılabilir çoklu sağlayıcı seçimi, bağısız i18n soyutlaması ve sahtecilik önleyici test garantileri sunar.

Kısa açıklama

Birden fazla LLM destekli CLI ajanını oluşturup yönetmek için tek bir birleşik arayüz sağlayan, yeniden kullanılabilir bir Go modülü. Hibrit boru+dosya protokolü üzerinden çalışan, devre kesicili ve seçilebilir yönlendirme stratejilerine sahip iş parçacığı güvenli ajan havuzu. Kasıtlı olarak tüketiciye bağımlı olmayan yapıda, takılabilir i18n çevirici ile birlikte gelir.

Uzun açıklama

LLMOrchestrator, bağısız CLI kodlama ajanlarını düzenlemek için ortak bir altyapıdır — çoklu ajan sistemlerinin sessizce ihtiyaç duyduğu ve genellikle kötü şekilde yeniden inşa ettiği temel yapı. OpenCode, Claude Code, Gemini CLI, Junie ve Qwen Code gibi araçlar için süreç oluşturma, mesaj çerçevelendirme ve sonuç ayrıştırma işlemlerini her proje yeniden uygulamak yerine, tek bir birleşik Agent arayüzü, iş parçacığı güvenli AgentPool ve birden fazla sağlayıcıyı tek bir arayüz altında toplayan MultiProviderPool sunar. Yönlendirme, AgentSelector aracılığıyla takılabilir hale getirilmiştir — gereksinimleri karşılamayan sağlayıcıları atlayan döngüsel dağıtım veya öncelik sıralı yedekleme — böylece işin nasıl dağıtılacağı sizin belirleyeceğiniz bir politika olur, sabit

kodlanmış bir varsayım değil. Her somut ajan, süreç yaşam döngüsünün tamamını yöneten ortak bir BaseAdapter üzerinden ince bir adaptördür: boru kurulumuyla başlayan, SIGTERM ardından SIGKILL ile nazikçe durdurma, yeniden başlatma ve canlılık kontrolü — karmaşık ve hataya açık kısımlar, tek seferde çözülmüş.

İletişim kasıtlı olarak hibrittir, işe uygun taşıma yöntemi kullanılır. Boru taşıma, hızlı etkileşimli mesajlaşma için satır sonu ile ayrılmış JSON iletilerini, isteğe bağlı okuma zaman aşımı ve yanıt uzunluğu sınırıyla taşırken; dosya taşıma, boruda yaşamaması gereken büyük veya kalıcı veriler için oturum başına gelen/giden/kullanılan izinler kullanır. Dayanıklılık sonradan akla gelen bir düşünce değil, yapısaldır: her ajan için devre kesici, üç ardışık başarısızlıktan sonra 60 saniyelik soğuma süresi için açılır ve yarı açık durumda bir deneme yapılır; arka planda çalışan bir sağlık monitörü ajanları sürekli kontrol eder, böylece çökmüş bir ajan gelen trafik beklemeden kendini toparlayabilir. Havuzdan ajan alma işlemi, CPU'yu meşgul eden bir bekleme döngüsü yerine koşul değişkeni üzerinde bloke olur ve yanıt ayrıştırıcısı durum bilgisi taşımayan, eşzamanlı çağrılara güvenli bir yapıdadır. Modül kesinlikle bağımsızdır — tüketiciye özgü hiçbir detay sızmaz — ve tüm kullanıcıya yönelik metinler, eksik çevirinin fark edilmesini sağlayan NoopTranslator ile mesaj kimliklerini aynen döndüren takılabilir bir i18n Translator üzerinden geçer.

Neden geliştirdik

Her çoklu ajan sistemi, CLI ajanlarını güvenilir şekilde başlatmak ve onlarla iletişim kurmak zorundadır. Süreç oluşturma, çerçevelendirme, ayrıştırma ve hata yönetimini her projede yeniden çözmek hem verimsiz hem de hataya açıktır. LLMOrchestrator, bu işlevleri tek bir bağımsız, yeniden kullanılabilir modülde toplar; uzmanlaşmış sorumluluğu sayesinde yeniden kullanılabilir hale gelir — ve bu yeniden kullanılabilirlik, tüketiciye özgü herhangi bir detay sızarsa yok olur.

İçerik

Neden Oyunun Kurallarını Değiştiriyor?

“Farklı CLI ajanlarından oluşan bir orduyu yönetmek” artık projeye özel zahmetli bir mühendislik uğraşı olmaktan çıkıyor; tek bir kütüphane içe aktarımıyla havuzlama, devre kesici, yaşam döngüsü yönetimi ve takılabilir yönlendirme gibi sorunlar çözülmüş ve sağlamlaştırılmış halde geliyor. Üstelik “derleniyor” demekle yetinmeyip gerçek sistemi uçtan uca test eden blöf

karşıtı testler sayesinde, eşzamanlılık ve hata durumlarında da güvenebileceğiniz bir soyutlama elde ediyorsunuz — sadece şemada doğru görünen değil.

Yenilikçi Yönleri

- **Hibrit boru+dosya protokolü** — etkileşimli hız (JSON satırları stdin/stdout üzerinden, okuma zaman aşımı, yanıt sınırları) ve büyük veri aktarımları için dayanıklı dosya tabanlı değişim (gelen/giden/ortak klasörler), böylece gecikmeyi dayanıklılıkla takas etmeniz gerekmez.
- **Takılabilir seçicilerle çok sağlayıcı havuz** — birçok CLI sağlayıcısı için tek bir arayüz, yönlendirme politikası olarak yuvarlak masa veya tercih sıralaması seçimi, kodun içine gömülü değil.
- **Ajan başına devre kesici + arka planda sağlık izleme** — otomatik bozulma ve kurtarma (3 hata → 60 saniye açık → yarı açık deneme), böylece sorunlu bir ajan izole edilir ve manuel müdahale olmadan sessizce geri alınır.
- **Boşta beklemeyen havuzlama** — Acquire, eşleşen sağlıklı bir ajan boşalana veya bağlam iptal edilene kadar sync.Cond üzerinde bloklanır, böylece beklemenin CPU maliyeti sıfırdır.
- **Sıkı ayrıştırma + blöf karşıtı uluslararasılaştırma** — NoopTranslator, mesaj kimliklerini aynen döndürür, böylece eksik çeviri gözden kaçmaz, sessizce boşluk bırakmaz.
- **Güvenlik varsayılana** — ikili dosya yolu izin listesi, kabuk yerleştirmeyi ve dolayısıyla komut enjeksiyonu yüzeyini ortadan kaldırır; yol geçişi koruması, 1 MiB yanıt sınırı (kontrolsüz çıktıya karşı) ve günlüklerde API anahtar maskeleyiyle desteklenir.
- **Blöf Karşıtı Zorluk aracı** — beş farklı dilde gerçek disk/JSON/ayrıştırıcı döngüleri, özelliğin bozulduğunda sıfırdan farklı çıkış yapması gereken eşlenik bir mutasyon kapısıyla — sistemin gerçekten başarısız olabileceğini kanıtlayan bir test.

En Büyük Teknik Zorluklar ve Çözümleri

- **Güvenilir ajan süreci G/Ç.** Başlatılan bir CLI süreciyle iletişim kurmak aldatıcı derecede zordur; hibrit boru+dosya taşıma, mesaj/ayrıştırıcı sözleşmesiyle iki tarafın da kablo formatında anlaşması ve tüm süreç yaşam döngüsünü merkezi hale getiren bir BaseAdapter ile çözüldü — SIGTERM zaman aşımına nazik geçiş ve SIGKILL yedekleme dahil.
- **Boşta beklemeden eşzamanlılık.** Acquire'ın, yetenekleri eşleşen bir ajan gerçekten boşalana kadar uyuduğu bir mutex + koşul değişkeni AgentPool ile çözüldü; yan etkisiz, durum

bilgisi olmayan bir ayrıştırıcı da birçok goroutine'den aynı anda güvenle çağrılabilir.

- **Sağlayıcı hatası izolasyonu.** Bir sağlayıcının diğerlerini etkilememesi için çözüldü: ajan başına devre kesiciler patlama yarıçapını sınırlar, sağlık izleme goroutine'i ise istek gelmese bile kurtarmayı sürdürür.
- **Sadece derlenme değil, doğruluğun kanıtlanması.** Zorluk çalıştırıcısıyla çözüldü: en/sr/ja/es/de dillerinde gerçek sistemi çalıştıran düzinelerce değişmez, artı özelliği kasten bozan bir mutasyon kapısı (LLMORCH_MUTATE_RUNNER=1 başarısız olmalı → sarıcı çıkış kodu 99) — kapının kendisinin blöf olmadığını kanıtlar.
- **Sessiz başarısızlık olmadan yerelleştirme.** Mesaj kimliklerini aynen döndüren NoopTranslator arayüzü ve tüketici başına çevirmen enjeksiyonuyla çözüldü — çeviri eksikliği her zaman görünür, örtbas edilmez.

İçerik

Teknoloji Yığını

- **Go (1.25)** — birinci sınıf eşzamanlılık ve temiz süreç yönetimi sunduğu için tercih edildi; bu da canlı ajan süreçlerinin orkestrasyonu için tam olarak gereken özellikler. Modülün kendisi, ajan adaptörleri, taşıma katmanları ve ayrıştırıcıyı içerir.
- **Go standart kütüphanesi (+ testify, yaml.v3)** — bağımlılık yüzeyini minimumda tutmak ve *hiçbir* LLM SDK'sını dahil etmemek için bilinçli bir seçim; böylece modül hafif kalır ve herhangi bir tüketiciye gömülebilir, satıcı bağımlılıkları taşınmaz.
- **Boru taşıma katmanı (stdio üzerinden JSON-satırları)** — hızlı etkileşimli mesajlaşma için seçildi; okuma zaman aşımı ve yanıt uzunluğu sınırlarıyla güçlendirildi, böylece takılı kalan veya kontrolden çıkmış bir ajan çağırana tarafı bloke edemez.
- **Dosya taşıma katmanı (gelen/giden/paylaşılan)** — oturum başına dayanıklı ve büyük veri alışverişi için seçildi; burada boru kullanımı yanlış bir tercih olurdu.
- **sync.Mutex/sync.Cond** — bloke edici, adil ajan havuzu edinimi için seçildi; meşgul bekleme olmadan uygulanır.
- **Devre kesici + Sağlık İzleyici** — yalnızca hata tespiti değil, aynı zamanda ajan başına dayanıklılık ve aktif kurtarma sağlamak için birlikte seçildi.
- **pkg/i18n Çevirmen** — tüketiciye özgü dizelerin çekirdekten ayrı tutulmasını sağlayan, bağımsız yerelleştirme arayüzü olarak tercih edildi.
- **Zorluk test ortamı (challenges/runner) + Makefile (test -race, fuzz, cover)** — blöf karşıtı, kanıta dayalı doğrulama için seçildi; yarış koşulu tespiti ve ayrıştırıcı fuzzing testleriyle,

doğruluğun varsayılmak yerine düşmanca koşullar altında kanıtlanması sağlanır.

Durum ve Dürüstlük Notları

- **Durum: beta.** Birden fazla Helix/vasic projesinde alt modül olarak kullanılan, bağımsız ve yeniden kullanılabilir bir modül.
Lisans: Apache-2.0; GitHub deposu herkese açık.
- Model meta verileri HelixQA üzerinden köprülenen LLMsVerifier'dan gelir; bu modül LLMsVerifier/VisionEngine/DocProcessor'u doğrudan içe aktarmaz. Üst uygulamanın CLAUDE.md dosyasında (Gin/PostgreSQL vb.) bahsedilen yığınlar helix_code ile ilgilidir, bu modülle değil.

Öncelik seviyesi: Helix-öncelikli (LLM-altyapı kümesi — bağımsız yeniden kullanılabilir modül). HelixTrack'dan sonra gelir.